

ADVANCED TOPICS

Using evaluations with the Kustomer API

Using evaluations with the Kustomer API

Use the **AI Evaluations API** to programmatically create, execute, and analyze evaluation runs for **AI Automations**.

Evaluations simulate customer scenarios and measure whether an AI Automation behaves correctly and consistently. Each evaluation runs structured **test cases** against a full automation workflow and returns detailed execution results.

This API allows you to:

- Create evaluation test scenarios
- Run evaluations against an AI Automation
- Inspect execution traces and grading results
- Measure reliability across multiple runs

To ensure quality service and prevent abuse, Kustomer [limits the number of API requests](#) that can be made within a short time.

Defining AI Automations

AI Automations (formerly **AI Agent Teams**) are orchestrated AI systems that coordinate multiple specialized AI agents to resolve customer requests.

An AI Automation may include:

- A **supervisor agent** that routes tasks.
- Multiple **specialized agents**.
- **Tools** that allow agents to access systems and perform actions.
- Knowledge sources agents access to provide support.

Evaluations execute against the **entire AI Automation workflow**, not individual agents.

Only **AI Automations** are supported by the **Evaluations API**.

Workflows are a separate automation system that performs event-based actions within the Kustomer platform.

Evaluations Overview

AI systems generate probabilistic responses. The same input can produce different outputs across executions.

Evaluations help measure:

- **Accuracy** – Whether the automation reaches the correct outcome
- **Reliability** – Whether behavior remains consistent across runs

Each evaluation contains:

- Up to **30 test cases**
- Up to **25 executions per test case**

Each execution produces:

- Pass / Fail outcome
- Conversation trace
- Tool usage
- Error information
- Execution timing

Evaluations operate only on **test data**, not live customer interactions, using these core objects:

Automation

Represents the **AI Automation** being evaluated.

You must provide an **automation ID** when creating an evaluation.

Evaluation

An **evaluation** groups multiple test cases for an automation.

An evaluation defines:

- The automation being tested
- Configuration for evaluation runs
- A set of test cases

Evaluation Test Case

A **test case** defines a simulated customer scenario.

Each test case includes:

- Customer profile used for the simulation
- Input prompt or conversation context
- Expected response criteria
- Tool usage requirements (optional)
- Agent invocation requirements (optional)

Evaluation Run

An **evaluation run** executes one or more test cases against the automation.

Each run may execute test cases multiple times to detect behavioral variation.

Executing evaluations by order of operations

The following sequence describes how to run an evaluation through the API.

1. Get automation ID
2. Create evaluation
3. Create evaluation test cases
4. Execute evaluation
5. Retrieve evaluation results

1. Retrieving an Automation ID

Before creating an evaluation, obtain the **automation ID** for the AI Automation you want to test.

This ID is required when creating the evaluation.

Example:

```
GET /automations
```

Save:

```
automation_id
```

2. Creating an Evaluation

Create an evaluation associated with the automation. `automation_id` is a required value.

POST `/evaluations`

Example request:

```
{
  "automationId": "automation_id",
  "name": "Refund workflow evaluation"
}
```

Example response:

```
{
  "id": "evaluation_id",
  "automationId": "automation_id"
}
```

Save:

`evaluation_id`

3. Creating Evaluation Test Cases

Add a test case or multiple test cases to the evaluation.

POST `/evaluation-test-cases`

Each evaluation supports **up to 30 test cases**.

Example:

```
{
  "evaluationId": "evaluation_id",
  "name": "Refund request scenario",
  "customerId": "customer_id",
  "testUserMessage": "I want a refund for my last order",
  "expectedResponse": "Provide refund instructions and confirm eligibility"
}
```

Save:

test_case_id

Test Case Configuration

The following attributes may be defined when creating a test case.

Field	Description
name	The name of the test case. Required.
input	Test case input data. Required. For example, "I want to refund my order."
conversationId	Source conversation ID for multi-turn context.
messageId	Source message ID for multi-turn context.
resourceId	Associated resourceId . Required.
resourceType	Resource type (Currently only customer is allowed).

Tool Requirements

Tools specified in the test case ensure the correct system integrations are used during evaluation.

Example:

```
"requiredTools": [  
  "order_lookup",  
  "refund_processor"  
]
```

The evaluation engine checks whether the specified tools were invoked during execution.

4. Running an Evaluation

There are **two POST endpoints** associated with running evaluations.

Although both initiate execution, they operate at different scopes.

Running an Entire Evaluation

Execute this endpoint to run **all test cases** associated with an evaluation.

```
POST /evaluations/{evaluation_id}/run
```

This endpoint:

- Executes all test cases
- Runs each test case multiple times
- Produces aggregated results

Use this endpoint when validating an automation before deployment.

Run a Specific Test Case

Runs **one test case** independently.

```
POST /evaluation-test-cases/{test_case_id}/run
```

Use this endpoint for:

- Debugging a single scenario
- Iterating on a specific prompt
- Testing agent behavior quickly

Retrieving Results

Evaluation execution produces structured results that can be retrieved via API.

There are two evaluations test case results endpoints:

Although both endpoints return similar data, they serve different purposes.

Endpoint	Purpose
List Results	Retrieve multiple results and discover result IDs
Result by ID	Retrieve detailed information for a specific execution

In most workflows:

1. Call **List Results**
2. Select a specific result
3. Retrieve detailed run data with **Result by ID**

List Evaluation Test Case Results

```
GET /evaluation-test-case-results
```

Returns a list of results for executed test cases.

Example response:

```
[
  {
    "id": "result_id",
    "evaluationId": "evaluation_id",
    "testCaseId": "test_case_id",
    "status": "pass",
    "executionTime": 4.2
  }
]
```

Use this endpoint to:

- Retrieve all results for an evaluation
- Obtain result IDs
- Review pass/fail outcomes

Get Evaluation Test Case Result by ID

```
GET /evaluation-test-case-results/{result_id}
```

Returns detailed information for a specific result.

Example response:

```
{
  "id": "result_id",
  "testCaseId": "test_case_id",
  "status": "pass",
  "conversationTrace": [],
  "toolCalls": [],
  "executionTime": 4.2
}
```

Example Evaluation Workflow

```
GET /automations
```

```
POST /evaluations
```

```
POST /evaluation-test-cases
```

```
POST /evaluations/{evaluation_id}/run
```

```
GET /evaluation-test-case-results
```

Optional:

```
GET /evaluation-test-case-results/{result_id}
```

Best Practices

Run multiple executions

Because AI responses vary, run evaluations multiple times to detect inconsistent behavior.

Test multiple phrasings

Create test case variants to simulate how different customers might ask the same question.

Examples:

```
Where is my order?  
Has my order shipped?  
Can you track my order?
```

Validate tool usage

If an automation must call specific systems (for example order lookup APIs), include those tools as required criteria in test cases.

Evaluate after configuration changes

Run evaluations whenever you modify:

- Agent instructions
- Knowledge sources
- Tool definitions
- Automation routing logic

Limitations

- Maximum 30 test cases per evaluation
- Maximum 25 runs per test case
- Evaluations run only against AI Automations

 Updated 1 minute ago

[← OAuth 2.0 Provider](#)

[App usability guidelines →](#)

Did this page help you?  **Yes**  **No**